

Document d'architecture technique

Table des matières

Infrastructure globale.....	4
Schéma.....	4
Table des machines.....	5
Active Directory Domain Services	6
WSUS	7
PKI	8
VPN Client-to-Site	10
Service de Backup	12
SIEM	13
EPP & EDR	15
Courriels.....	16
Mail Gateway	17
Mailbox Server	18
Postes utilisateurs mails.....	21
Site vitrine	22
Serveur Reverse Proxy	22
Serveur Web	23
DNS.....	24
DNS Autoritaires	26
DNS Maitre.....	26
DNS Esclave	26
DNS Récursifs	27
DNS Résolveur interne	27
DNS Résolveur externe	27
Proxy	28
Inspection HTTPS (SSL bumping)	28
Modes de fonctionnement.....	29
Fichiers de configuration principaux.....	29
Flux réseau OPNsense (firewall interne).....	30
HTTP Explicite (port 3128) flux détaillés	31

HTTP Transparent (port 3127) flux détaillés	31
HTTPS explicite (port 3128) en SSL bumping flux détaillés	32
HTTPS transparent (port 3129) en SSL bumping flux détaillés.....	33
Bastion (abandonné)	34
Choix techniques.....	34
Fonctionnement	35
Fichiers de configuration principaux.....	35
Ansible	36
Rôle.....	36
Choix techniques.....	36
Fonctionnement	36
Fichiers de configuration principaux.....	37
Flux réseau	37
Serveur de fichiers	38
Choix techniques.....	38
Fichiers de configuration principaux.....	38
Flux réseau	38
Miroir (abandonné)	39
Architecture mise en place	39
Fonctionnement	39
Causes de l'abandon	40
Retour en arrière.....	40
ERP ODOO.....	41
Choix techniques.....	41
Fichiers de configuration principaux.....	41
Flux réseau	41

Infrastructure globale

Schéma

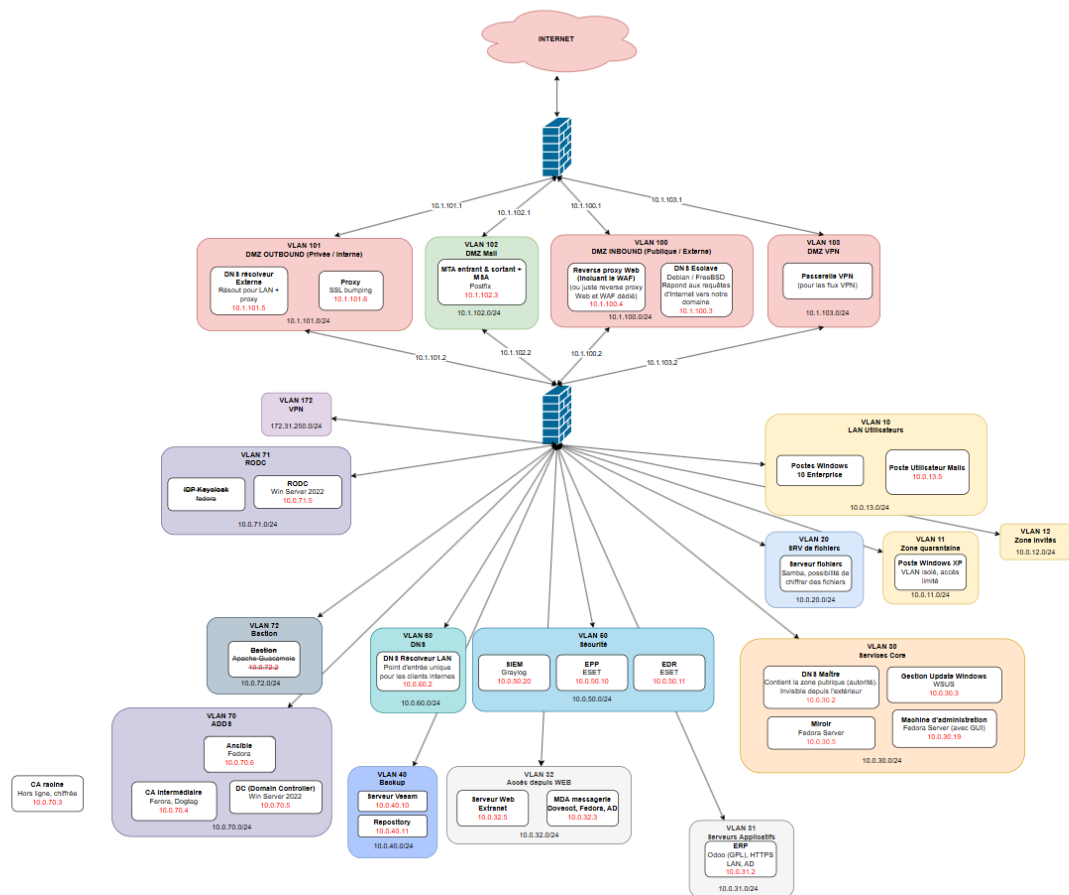


Table des machines

Service	IP	VLAN
CA Root	10.0.70.3	70
CA Int	10.0.70.4	70
DC	10.0.70.5	70
Parc linux (Ansible)	10.0.70.6	70
DNS esclave	10.1.100.3	100
Reverse Proxy	10.1.100.4	100
DNS résolveur interne	10.0.60.2	60
DNS résolveur externe	10.1.101.5	101
Proxy	10.1.101.6	101
Parc windows	10.0.13.5	13
Mail gateway	10.1.102.3	102
Mailbox server	10.0.32.3	32
Web server	10.0.32.5	32
DNS maître	10.0.30.2	30
Machine de test	10.0.30.14	30
Machine administration	10.0.30.19	30
VEEAM	10.0.40.10	40
Repo-backup	10.0.40.11	40
ESET-Protect	10.0.50.10	50
ESET-Inspect	10.0.50.11	50
Graylog	10.0.50.20	50
Bastion	10.0.72.2	72
Serveur de fichiers	10.0.20.10	20
ERP	10.0.31.2	31

Active Directory Domain Services

Composants

- Un contrôleur de Domaine (DC) Windows Server 2022
- Un contrôleur de Domaine en lecture seule (RODC) Windows Server 2022

Rôle : Qu'est-ce qu'Active Directory Domain Services (AD DS) ?

Active Directory Domain Services (AD DS) est le cœur de la gestion des identités et des accès dans un environnement Windows sur site. C'est le service centralisé qui permet à une entreprise de contrôler qui peut accéder à quoi, et dans quelles conditions.

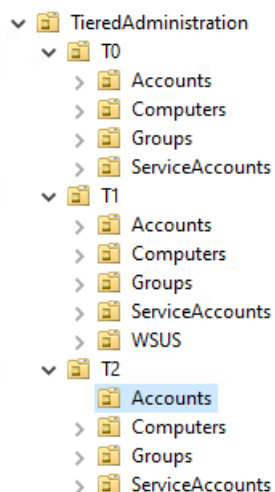
Ses principales fonctions sont les suivantes :

- Authentification centralisée des utilisateurs et des machines au sein du réseau Windows.
- Gestion unifiée des comptes, des groupes, des ordinateurs et des objets du réseau local (LAN) d'entreprise.
- Contrôle d'accès aux applications, serveurs de fichiers et ressources on-premises, via des politiques de sécurité intégrées.
- Application d'une configuration cohérente grâce aux GPOs (Group Policy Objects), qui permettent de définir automatiquement des règles sur l'ensemble des postes d'un domaine.

En résumé, AD DS constitue la colonne vertébrale des identités dans un réseau d'entreprise.

Structure

L'annuaire a été défini de manière à respecter un modèle en Tiers :



Flux firewall

Client Port(s)	Server Port	Service
49152-65535/UDP	123/UDP	W32Time
49152-65535/TCP	135/TCP	RPC Endpoint Mapper
49152-65535/TCP	464/TCP/UDP	Kerberos password change
49152-65535/TCP	49152-65535/TCP	RPC for LSA, SAM, NetLogon
49152-65535/TCP/UDP	389/TCP/UDP	LDAP
49152-65535/TCP	636/TCP	LDAP SSL
49152-65535/TCP	3268/TCP	LDAP GC
49152-65535/TCP	3269/TCP	LDAP GC SSL
53, 49152-65535/TCP/UDP	53/TCP/UDP	DNS
49152-65535/TCP	49152-65535/TCP	FRS RPC
49152-65535/TCP/UDP	88/TCP/UDP	Kerberos
49152-65535/TCP/UDP	445/TCP	SMB
49152-65535/TCP	49152-65535/TCP	DFSR RPC
49152-65535/TCP	9389/TCP	Active Directory Web Services (ADWS)

WSUS

Composants & rôle

- Un Windows Server 2022 avec WSUS installé dessus. Son rôle est de lister les mises à jour windows, sélectionner celles que l'on juge nécessaires avant de les approuver, les télécharger avant de les distribuer sur notre parc. Il remplace les serveurs Windows Update pour les machines sur notre LAN. On peut voir toutes les mises à jour possible sur notre parc grâce à des rapport d'état envoyé par nos postes utilisateurs et serveurs windows.

Flux

Machine	Port	
---------	------	--

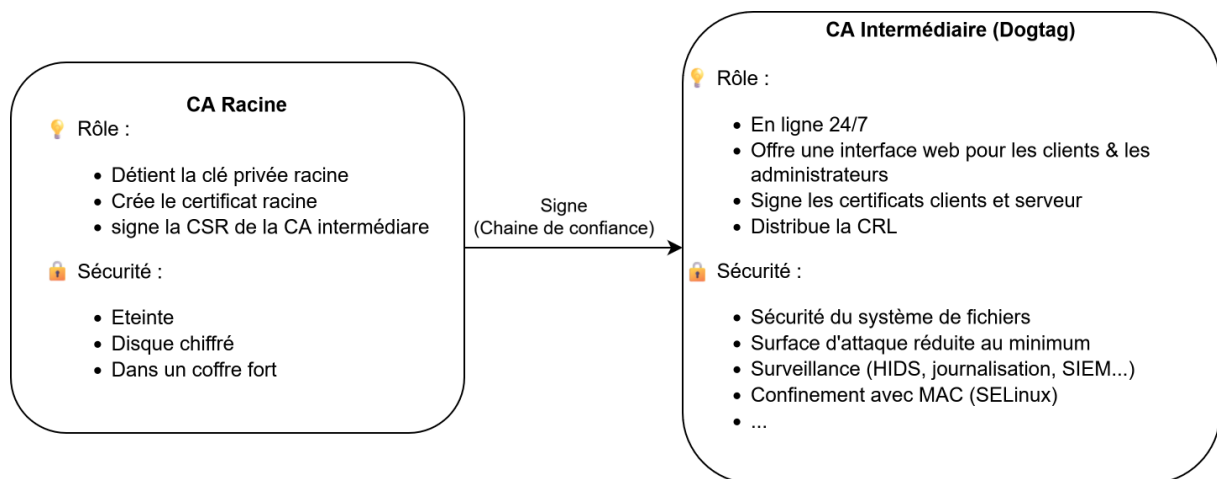
Serveur WSUS	8530	TCP/UDP : Autorise le flux http vers WSUS
Postes utilisateurs et serveurs	*	Tous les VLAN avec des serveurs windows ou postes utilisateurs ont le droit joindre WSUS

PKI

Composants & rôle de notre PKI

- **Autorité de Certification Racine (CA racine)** : le sommet de la pyramide de confiance. Sa clé privée est l'élément le plus critique, elle sert à signer des certificats AC intermédiaires. On suggère fortement de l'isoler hors ligne pour garantir une confidentialité maximale.
- **Autorité de Certification Intermédiaire (AC intermédiaire)** : signée par la racine et c'est elle qui délivre les certificats aux utilisateurs finaux ou aux équipements. Elle doit rester opérationnelle en continu pour répondre aux demandes.

Schéma d'infrastructure



(une bulle = une machine linux Fedora)

Services & Choix techniques

Composant	Technologie choisie	Rôle dans l'infrastructure PKI	Pourquoi cette technologie a-t-elle été choisie ?
-----------	---------------------	--------------------------------	---

Autorité de certification racine (Root CA)	OpenSSL	Ancre de confiance hors ligne de l'ensemble de l'infrastructure	Solution légère, éprouvée, vérifiable, présentant une surface d'attaque minimale, et offrant un excellent contrôle des politiques de certificats et des opérations de signature
Autorité de certification intermédiaire (ICA)	Dogtag Certificate System	Autorité de certification en ligne pour les opérations quotidiennes liées aux certificats	Plateforme PKI d'entreprise avec gestion du cycle de vie des certificats, profils, approbations, révocation, OCSP et capacités d'automatisation
Service d'annuaire pour Dogtag	389 Directory Server	Stockage interne des métadonnées/configurations Dogtag	Intégration native de Dogtag, architecture entièrement prise en charge, évite le couplage des composants internes de l'autorité de certification à AD

Flux firewall

Source	Client Port	Server Port
Machine d'administration	*	8443/TCP

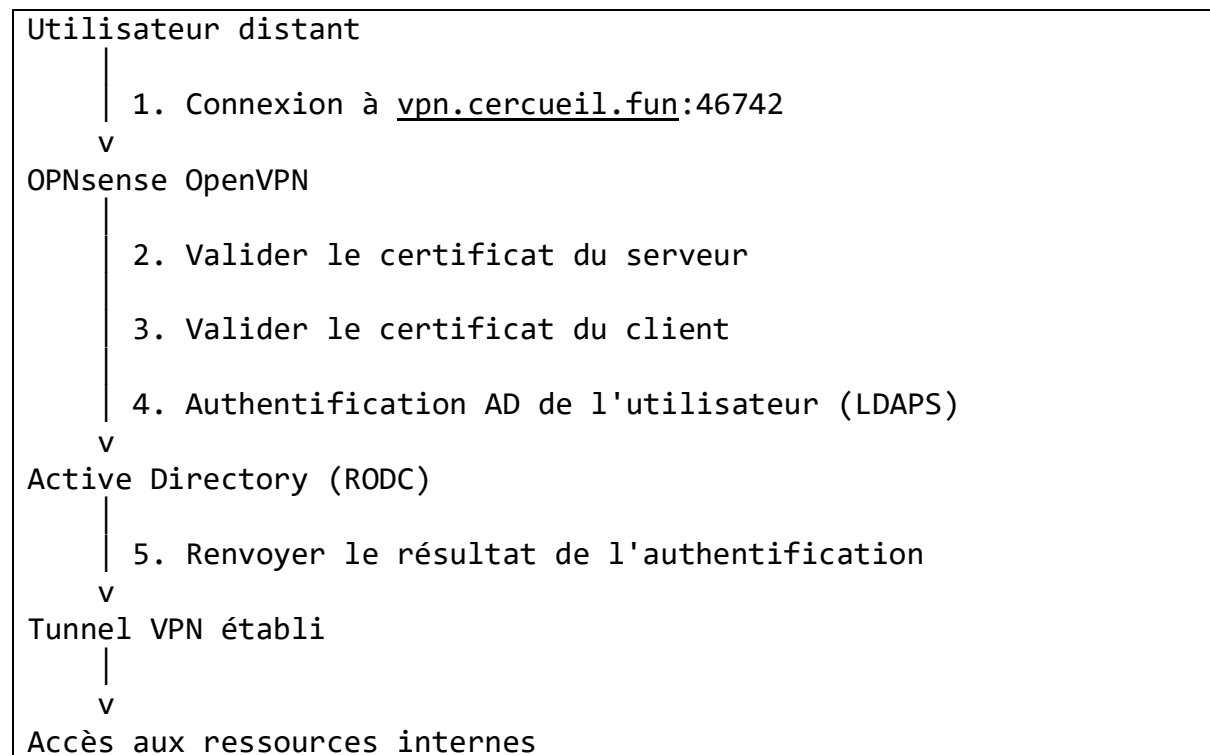
Flux pour accéder à l'interface web d'administration depuis la machine d'admin

VPN Client-to-Site

Rôle

Permettre aux clients d'accéder aux ressources des différents LANs à distance.

Schéma d'infrastructure



Services & Choix techniques

Composant	Technologie choisie	Rôle	Raison de ce choix
Protocole VPN	OpenVPN	Crée le tunnel chiffré entre les utilisateurs distants et le réseau interne	Prise en charge native dans OPNsense, écosystème mature, authentification par certificat, large compatibilité avec les clients
Passerelle VPN	Firewall interne OPNsense	Termine les sessions VPN et applique les politiques réseau	Déjà déployé en tant que pare-feu interne, administration centralisée. Point de contrôle unique pour l'accès, le pare-feu et la journalisation

Authentification des utilisateurs	Active Directory via LDAP(S)	Vérifie l'identité des utilisateurs (nom d'utilisateur/mot de passe)	Réutilise les identités d'entreprise et les processus de cycle de vie existants. Gestion centralisée des comptes, désactivation immédiate lorsque les utilisateurs quittent l'entreprise
Certificats client / serveur X.509 émis par l'infrastructure PKI interne	Dogtag Certificate System	Fournit une confiance au niveau des machines (et en plus des identifiants de l'utilisateur)	Protection contre l'usurpation d'identité du VPN + Authentification TLS mutuelle

Flux Firewall

2 choses sont à prendre en compte :

1. Les routes créées vers les différents VLANs lors de l'initiation du tunnel (dans notre cas assez permissives, pour permettre l'administration de toutes les machines de l'infra par VPN)

Routing

Local Network

10.0.70.0/24 × 10.0.72.0/24 × 10.0.60.0/24 ×
 10.0.71.0/24 × 10.0.50.0/24 × 10.0.40.0/24 ×
 10.0.32.0/24 × 10.0.31.0/24 × 10.0.30.0/24 ×
 10.0.20.0/24 × 10.0.13.5/24 × 10.1.101.0/24 ×
 10.1.102.0/24 × 10.1.100.0/24 ×

Clear All Copy Paste Text

Remote Network

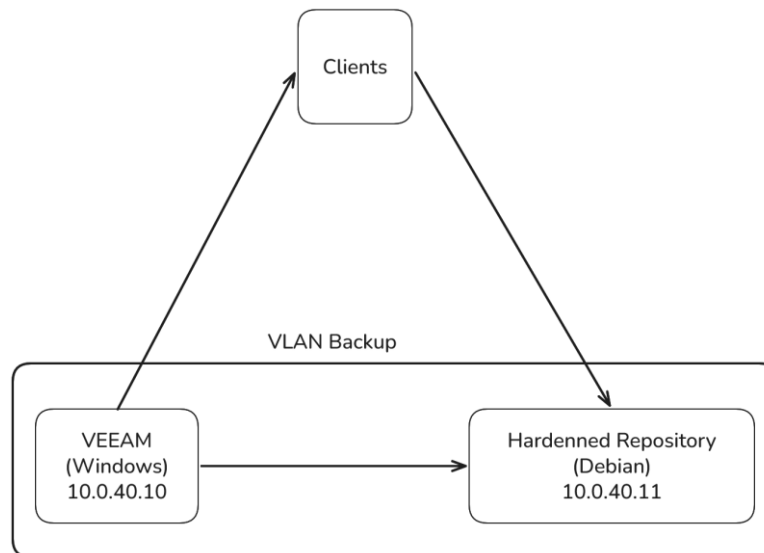
Clear All Copy Paste Text

2. Les règles firewall pour autoriser et contrôler précisément les flux depuis le VLAN OpenVPN Network vers les machines de l'infra

Source	Port	Destination
OpenVPN Network	22/TCP	*
OpenVPN Network	443/TCP	Firewall OPNsense

Service de Backup

Les backups sont réalisés avec l'application VEEAM Backup & Replication Community Edition. Le serveur VEEAM est une machine Windows. Les sauvegardes sont stockées dans un hardenned repository, sur une machine Debian.



Le serveur Veeam contrôle les agents et le repository à distance. Lorsqu'un backup a lieu, le client envoie directement les données au repository

Flux réseau :

Source	Protocol	Ports	Destination	Ports
VEEAM	TCP	*	Clients (Windows)	135, 445, 2500:3300, 6160, 6162, 6184, 10001, 49162:65535
Clients (Windows)	TCP	*	VEEAM	2500:3300, 6160, 6162, 9392, 10002
Clients (Windows)	TCP	*	Repo-backup	2500:3300, 6162,

SIEM

Le SIEM est basé sur 3 services : MongoDB (garde en mémoire nos configurations), OpenSearch (notre moteur de recherche de logs) et Graylog (notre serveur d'application). Ces trois services tournent sur un seul serveur Fedora. La connexion à l'interface web se fait sur le port 9000.

Les clients Linux envoient leurs logs via Rsyslog. Le fichier de de base configuration contient :

```
*.* action(type="omfwd" target="10.0.50.20" port="1514" protocol="UDP"
template="RSYSLOG_SyslogProtocol23Format")
```

Ce dernier est ajusté en fonction du service et des logs que l'on veut recevoir ou non.

Inputs :

Nos logs arrivent sur graylog via 3 inputs différents :

- Un premier utilisé par Sidecar Beats via le port 5044 pour les postes windows
- Un second utilisé par Syslog via le port 1514
- Un dernier flux pour l'AD (en cours)

Streams :

Nos logs sont répartis dans 6 streams différents :

Debian	Les serveurs Debian
Default Stream Default	Contains messages that are not explicitly routed to other streams
Domain Controller	AD et RODC
Fedora	Les serveurs Fedora
OPNsense	flux firewall
Windows	Les postes windows

Index sets :

Pour garantir une bonne gestion de la rétention des données, nous avons segmenté les logs par « index sets ». Chaque catégorie de machine listée ci-dessus dispose de son propre index.

Index set	Préfixe	Rétention max
AD_index	ad_	90 jours
Firewall_index	opensense_	20 jours
Windows_index	windows_	40 jours
Debian_index	debian_	40 jours
Fedora_index	index_	40 jours

Nous avons ajusté les durées à 90 jours pour l'AD (car il est le cœur de notre sécurité), et 20 jours pour les pare-feux (afin de limiter la saturation du disque).

Notifications :

À ce jour (23/06/2026) nous avons configuré une alerte nous notifiant en temps réel de la connexion permanente d'un utilisateur en root sur l'un de nos serveurs. (D'autres alertes sont prévues d'ici la soutenance.) Cette dernière s'envoie avec un webhook discord et arrive dans un salon dédié de notre serveur d'équipe :



Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
Serveurs Linux	TCP	*	Graylog	1514	Syslog
Postes Windows	TCP	*	Graylog	5044,9000	Beats
Machine admin	TCP	*	Graylog	9000	Accès web

EPP & EDR

La protection des Endpoint est assurée par ESET Protect (EPP) et ESET Inspect (EDR).

Nous avons utilisé l'appliance virtuelle fournie par l'éditeur pour le serveur Protect. Pour le serveur Inspect, nous avons installé l'outil sur un serveur Windows. Sur les clients, il faut installer dans un premier temps l'agent de communication qui permet de contacter les serveurs. Dans un second temps, il faut déployer Endpoint Security et Inspect Connector depuis la console d'administration.

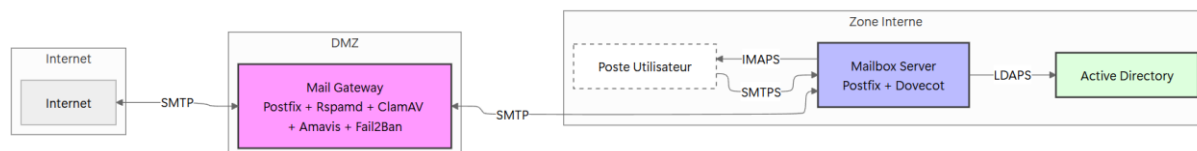
Flux réseau :

Source	Protocol	Ports	Destination	Ports
Clients	TCP/UDP	*	ESET-Protect	2222, 8883
Machine d'administration	TCP	*	ESET-Protect ESET-Inspect	443
Clients	TCP/UDP	*	ESET-Inspect	8093
ESET-Inspect	TCP/UDP	*	ESET-Inspect	2223

Courriels

Schéma :

L'architecture de notre service de messagerie repose sur une séparation en deux couches (Mail gateway et Mailbox server) pour limiter la surface d'attaque.



Choix techniques :

Composant	Solution	Justification
MTA	Postfix	Standard, robustesse
MDA	Dovecot	Standard, haute performance, intégration native avec LDAP/AD (malgré un changement de syntaxe récent énorme)
Filtrage/Défense	Rspamd, ClamAV, SpamAssassin, Fail2Ban	Standards open-sources, défense en profondeur, protection contre la brute force
Annuaire	Active Directory	Centralisation des identités via LDAP/SSSD

Tableau de versions :

Service	Version
Postfix	3.10.3-3
Certbot	4.1.1-4
Dovecot	2.4.4-1
OpenLDAP Clients	2.6.13-1
SpamAssassin	4.0.1-7
ClamAV	1.4.4-1
Amavis	2.14.0

Protocoles mails de sécurité DNS :

Mise en place sur notre serveur DNS maître des protocoles SPF, DKIM, DMARC avec règles de filtrage strictes pour éviter l'usurpation d'identité et assurer la délivrabilité de nos courriels.

Ajoute du DNSSEC pour sécuriser la résolution de noms contre les attaques par empoisonnement, pour valider l'authenticité et la fiabilité de l'ensemble de notre infrastructure de messagerie.

Pour vérifier nos protocoles, nous avons utilisé les sites suivants. Ces derniers témoignent d'un excellent score !

- internet.nl
- mxtoolbox.com

Mail Gateway

Située en zone démilitarisée elle agit comme un bouclier.

Sur cette machine, Postfix est chargé de recevoir les mails entrants depuis internet, d'effectuer un premier filtrage (avec Rspamd et ClamAV) et de relayer les mails valides vers le serveur interne (Mailbox server).

Aucune boîte mail n'est hébergée ici. Donc en cas de compromission, aucune donnée utilisateur n'est exposée.

Certificat SSL :

La gestion des certificats est automatisée grâce au service Certbot de Let's Encrypt, C'est nécessaire pour garantir la sécurité et la fiabilité de nos échanges mails (évite les attaques de type MITM).

Filtrage mail :

Nos courriers sont traités avant d'être livrés. Pour cela, 3 technologies interviennent.

- Amavis, le "content filter" qui orchestre le filtrage en transmettant les mails à
- ClamAV (analyseur antivirus) et
- SpamAssassin (analyseur heuristique), pour enfin le retransmettre à Postfix.

Fichiers de configuration principaux :

- **/etc/postfix/main.cf** : le cœur du serveur, il contient les paramètres globaux (hostnames, réseaux autorisés, protocoles, restrictions)
- **/etc/postfix/master.cf** : le gestionnaire de processus, il définit quels services Postfix doit lancer (smtp, submission, smtps, pickup, cleanup) et avec quels arguments
- **/etc/postfix/transport** : la table d'aiguillage, elle définit vers quelle machine envoyer les courriels en fonction de leur domaine de destination

Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
--------	----------	-------	-------------	-------	-------------

Internet	TCP	25	Mail gateway	25	Réception des mails entrants (SMTP)
Mail gateway	TCP	25	Internet	25	Envoi des mails vers l'extérieur
Mail gateway	TCP	25	Mailbox server	25	Relais SMTP vers le serveur interne
Mail gateway	TCP	80/443	Internet	80/443	Renouvellement Certbot
Mail gateway	UDP	53	DNS maître	53	Résolution de noms (destinataires)

Mailbox Server

Située en zone interne.

Sur cette machine, Postfix reçoit le courrier de la passerelle et le transmet au stockage géré par Dovecot.

Ce dernier gère les accès IMAP/POP3, qui authentifient les utilisateurs via l'Active Directory (LDAPS), et qui vérifie l'existence de l'utilisateur avant l'acceptation du mail.

Fichiers de configuration principaux :

- **/etc/dovecot/dovecot.conf** : configuration globale (nous n'avons pas fait de sous-fichiers de configuration pour faciliter la maintenance)
- **/etc/dovecot/conf.d/auth-ldap.conf.ext** : le gestionnaire de processus, il définit quels services Postfix doit lancer (smtp, submission, smtps, pickup, cleanup) et avec quels arguments
- **/etc/postfix/main.cf** : configuration du routage local

Architecture de notre conf Dovecot

Stockage des mails :

Le groupe vmail est le propriétaire unique de tous les dossiers de mails sur le disque dur, permettant à Postfix et Dovecot d'y accéder en toute sécurité sans créer de comptes pour tous les utilisateurs.

```
mail_home = /srv/mail/%{user} # Structure hiérarchisée par user
mail_driver = sdbox # Format de stockage
mail_path = ~/mail # Emplacement relatif des mails dans le home
mail_uid = vmail # Utilisateur système propriétaire des mails
mail_gid = vmail # Groupe système propriétaire des mails
```

```
first_valid_uid = 5000
```

Authentication :

On utilise le protocole LDAPS pour chiffrer les informations qui transitent entre le serveur et l'AD.

```
ldap_uris = ldaps://dc01.cercueil.local:636
ldap_base = DC=cercueil,DC=local
ldap_auth_dn =
CN=svc_dovecot_ldap,OU=ServiceAccounts,OU=T1,OU=TieredAdministrati
on,DC=cercueil,DC=local
ldap_auth_dn_password = MDP
ldap_version = 3
```

Database d'authentification des utilisateurs :

Passdb cherche dans l'AD si l'identifiant existe, Userdb indique où trouver les données.

```
passdb ldap { filter =
    (|(mail=%{user})(userPrincipalName=%{user}))
    bind = yes
}
userdb ldap { filter =
    (|(mail=%{user})(userPrincipalName=%{user})) {
        home = /srv/mail/%{user}
        uid = vmmail
        gid = vmmail
    }
}
```

Chiffrement SSL/TLS :

On impose TLSv1.2 minimum et on précise les chiffrements les plus robustes.

```
ssl = required
ssl_cipher_list = ECDHE-ECDSA-AES128-GCM-SHA256:...
ssl_min_protocol = TLSv1.2
ssl_server {
    cert_file = /etc/pki/tls/certs/fullchain.pem # Cert publique
    key_file = /etc/pki/tls/private/dovecot.pem # Clé privée
    dh_file = /etc/pki/tls/certs/dhparams.pem # Diffie-Hellman
}
```

Services internes :

Le service LMTP assure le transfert local entre Postfix et Dovecot.

Le service auth permet le partage d'authentification entre Postfix et Dovecot.

Le service anvil limite le débit des connexions vers Dovecot (en cas de brute force).

```
service lmtp {
    user = vmmail
    process_min_avail = 5
    inet_listener lmtp {
        listen = 127.0.0.1
        port = 24
    }
}
service auth {
    unix_listener auth-userdb {
        mode = 0660
        user = vmmail
        group = vmmail
    }
    unix_listener /var/spool/postfix/private/auth {
        mode = 0600
        user = postfix
        group = postfix
    }
}
service anvil {
    unix_listener anvil {
        mode = 0660
        user = vmmail
        group = vmmail
    }
}
```

Nous avons créé un compte de services dédié exclusivement à la liaison entre Dovecot et Active directory : `svc_dovecot_ldap`. C'est au travers de ce dernier que Dovecot s'authentifie auprès de l'AD afin d'avoir le droit de parcourir l'annuaire :

```
PS C:\Users\Administrateur> get-aduser svc_dovecot_ldap

DistinguishedName : CN=svc_dovecot_ldap,OU=ServiceAccounts,OU=T1,OU=TieredAdministration,DC=cercueil,DC=local
Enabled           : True
GivenName        : svc_dovecot_ldap
Name             : svc_dovecot_ldap
ObjectClass      : user
ObjectGUID       : 9d36ea3c-b3a3-4af7-9059-08d0b068f170
SamAccountName   : svc_dovecot_ldap
SID              : S-1-5-21-2811781259-2151170523-2478755382-1117
Surname          :
UserPrincipalName : svc_dovecot_ldap@cercueil.local
```

Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
Mailbox server	TCP	25	Mail gateway	25	Autoriser les e-mails sortants (SMTP)
Mailbox server	TCP	636	Active directory	636	Requêtes LDAPS (Auth et UserDb)
Mailbox server	UDP	53	DNS maître	53	Résolution noms internes et AD

Postes utilisateurs mails

L'accès au mails se fait via Thunderbird.

Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
LAN Users	TCP	993	Mailbox server	993	Consultation (IMAPS)
LAN Users	TCP	465	Mailbox server	465	Envoi (SMTPS)

Site vitrine

Notre site web <https://cercueil.fun> est servi par 2 machines (Serveur web + Reverse proxy) pour isoler notre code source et nos fichiers sensibles de l'extérieur.

Schéma

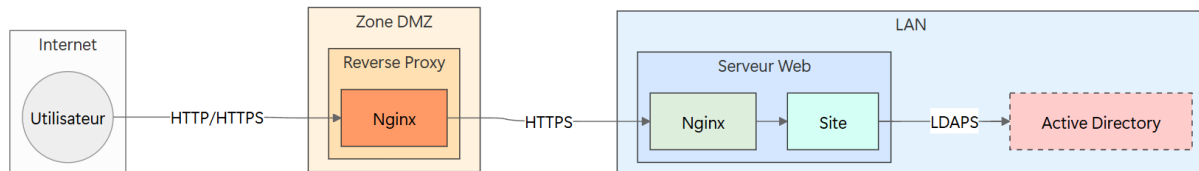


Tableau de versions :

Service	Version
MySQL	8.4.9
Node	22.22.2
Vue.js	3.5.32
Express	5.2.1
Nginx	1.30.2
Certbot	4.1.1-4
PM2	7.0.1

Serveur Reverse Proxy

Le Reverse Proxy est le point d'entrée unique du trafic web venant de l'extérieur vers notre site. Il protège notre serveur web hébergeant le code applicatif d'une exposition directe à Internet.

Choix techniques :

Technologie	Justification
Nginx	Choisi pour sa gestion native du protocole HTTP/2 et sa capacité à gérer des milliers de connexions simultanées
Certbot (Let's Encrypt)	Automatise la gestion des certificats, garantit que le HTTPS est toujours valide pour les utilisateurs
Fail2Ban	Rempart contre les attaques brute force, surveille les logs nginx pour bannir automatiquement les IP, efficace contre le DDOS.

Nous avons configuré Fail2Ban de manière à bannir automatiquement les IP qui tentent de se connecter de trop nombreuses fois à notre interface d'administration, ou les bots qui scannent notre site.

Nous avons inclus dans notre configuration Nginx des politiques CSP (Content Security Policy) robustes. Ces mesures neutralisent efficacement les attaques XSS et le détournement de contenu en contrôlant précisément les sources de données autorisées.

Grâce à ce durcissement, notre application atteint un score « Excellent » sur le site securityheaders.com !

Fichiers de configuration principaux

- `/etc/nginx/conf.d/cercueil.conf` : fichier définissant l'écoute sur les ports 80/443, la liaison avec les certificats Lets's Encrypt (via Certbot) ainsi que la redirection vers notre serveur web.
- `/etc/fail2ban/jail.local` : fichier de configuration des "prisons"
- `/etc/fail2ban/filter.d/nginx-ad-login` : Définition du filtre de bannissement pour la connexion AD.

Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
Internet	TCP	80, 443	Reverse proxy	80, 443	Accès public au site
Reverse proxy	TCP	80, 443	Serveur web	80, 443	Relais du trafic vers le serveur web

Serveur Web

À la suite d'attaques en début de P@8, le disque de cette machine a été chiffré.

Choix techniques :

Notre serveur web sert notre site grâce aux services suivants :

Solution	Justification
Nginx	Joue le rôle de serveur web local, sert les fichiers statiques de Vue.js et délègue les requêtes API vers Node.js
PM2	Pour faire tourner Node en continu et éviter l'arrêt du service suite à un crash du processus

Nous avons développé le site avec la stack suivante (elles sont en grande partie justifiées par le fait qu'elles sont connues par tous les membres de notre équipe) :

Composant	Solution	Justification
Frontend	Vue.js	Choisi pour sa réactivité et son approche basée sur les composants. Il rend notre catalogue de cercueils fluide sans rechargement de page

Backend	Node.js / Express	Nous avons besoin de gérer des requêtes API, Node est un standard moderne. Express rend l'API facile à maintenir
Base de données	MySQL	Pour un catalogue de produits, la structure relationnelle est incontournable. MySQL garantit l'intégrité de ces données, et utilise un utilisateur avec privilèges restreints en respectant le principe du moindre privilège.
Gestion de version	GitHub	Cette plateforme de collaboration nous permet de centraliser le code et ainsi faciliter le déploiement en continu.

Notre architecture est conçue avec les dépendances suivantes pour protéger l'intégrité de nos services à chaque étape :

- Délégation via **ldapjs** vers notre Active Directory pour une gestion unifiée des accès.
- Utilisation de **JWT** pour éliminer le besoin de gestion complexe des sessions en base de données.
- Filtrage des requêtes par **express-rate-limit** pour bloquer les attaques par force brute et le déni de service.
- Neutralisation des injections SQL par les requêtes préparées de **mysql2**.
- Isolation des variables sensibles via **dotenv**, garantissant qu'aucune donnée critique n'est exposée dans le code source.

Flux réseau :

Source	Protocol	Ports	Destination	Ports	Description
Reverse proxy	TCP	80, 443	Serveur web	80, 443	Réception des requêtes relayées
Serveur web	TCP	636	RODC	636	Auth LDAPS

DNS

Schéma

Le service DNS repose sur 5 serveurs différents au sein de notre infrastructure. Ils sont divisés en 2 types de serveurs :

- Autoritaires : sont responsable d'un domaine et répondent aux interrogations des résolveurs.
- Résolveurs : se chargent de trouver l'adresse IP des serveurs responsables du domaine demandé.

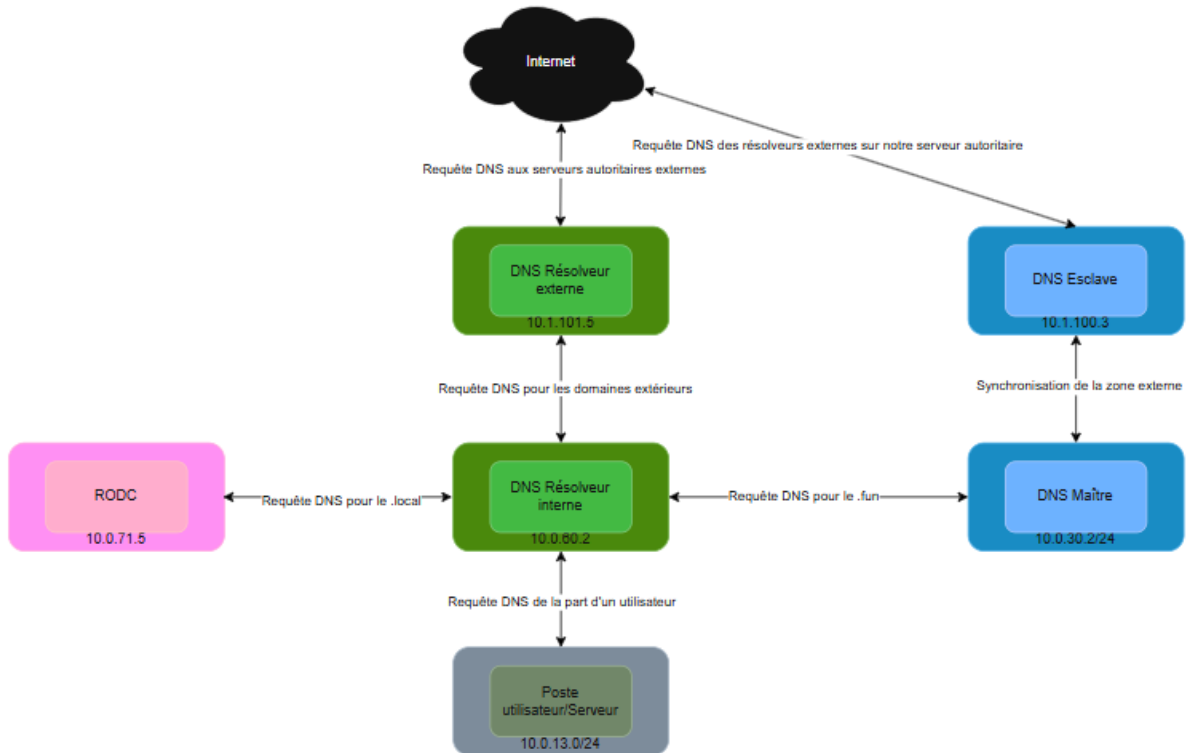


Schéma du fonctionnement des serveurs DNS

Choix techniques

Composant	OS	Solution	Justification
Autoritaires	Debian	Bind9	Robuste, standard de l'industrie, gère les zones complexes et les signatures dnssec
Résolveurs	Debian	Unbound	Léger, rapide et sécurisé, c'est aussi un standard.

Tableau des versions

Logiciel	Version
Bind9	9.20.23
Unbound	1.24.2

DNS Autoritaires

DNS Maître

Le DNS Maître est le serveur DNS autoritaires sur notre domaine "cercueil.fun". Il est situé dans notre LAN dans le VLAN Core en avec l'adresse IP "10.0.30.2".

C'est lui qui gère les différents enregistrements DNS, les configurations et autre qui définissent nos différents services joignables sur le LAN et depuis internet.

Fichiers de configuration principaux

- /etc/bind/named.conf : fichier incluant et liant tous les fichiers de configurations.
- /etc/bind/named.conf/local : fichier définissant les différentes vues : interne et externe. La vue interne est interrogeable par le résolveur interne donc toutes les machines des LAN alors que la vue externe est téléchargée par le DNS Esclave et sera celle disponible à la vue de tous sur internet. Dans ce fichier on définit l'utilisation du DNSSEC, qui a le droit d'interroger le maître, le fait de notifier l'esclave au changement du fichier de zone externe et le fait de le laisser télécharger le fichier ...
- /var/cache/bind/db.cercueil.fun.internal : enregistrement interne du domaine "cercueil.fun", permet de définir des serveurs type "mailbox.cercueil.fun" avec une adresse IP privée sans l'exposer sur internet.
- /var/cache/bind/db.cercueil.fun.external : enregistrement externe du domaine "cercueil.fun", disponible à tous il définit les différents serveurs joignables par tout le monde (serveur web, mail, DNS ...).

DNS ESCLAVE

Fichiers de configuration principaux

- /etc/bind/named.conf.local : fichier définissant le comportement du serveur. Il va préciser au serveur que celui-ci est l'esclave sur le domaine "cercueil.fun", désactiver la récursion pour éviter que des gens malveillants utilisent ce serveur comme rebond pour interroger des domaines. On peut aussi définir certaines sécurités comme désactiver la possibilité de télécharger la zone entière via une requête AXFR comme l'esclave le fait vers le maître ou encore cacher la version de bind utilisée.

DNS Récursifs

DNS RESOLVEUR INTERNE

Le DNS Résolveur interne est le serveur DNS de tous les serveurs et postes de notre réseau LAN. Il réoriente les requêtes DNS en fonction de règles que nous avons défini. Il redirige les requêtes à destination de “cercueil.fun” vers le DNS maître et a donc accès à la vue interne, les requêtes à destination de “cercueil.local” vers le RODC et les autres requêtes vers le résolveur externe.

Fichiers de configuration

- /etc/unbound/unbound.conf : C'est le fichier de configuration principal d'Unbound. Contrairement à Bind9, son rôle n'est pas de stocker des annuaires de machines, mais de gérer les accès récursifs (la recherche sur Internet) et de définir les règles de routage des requêtes DNS internes.

DNS RESOLVEUR EXTERNE

Fichiers de configuration

- /etc/unbound/unbound.conf : On autorise les requêtes de la part du proxy qui est dans le même vlan et du résolveur interne pour ensuite aller chercher sur internet.

Proxy

Le proxy est le point de passage obligatoire de tous les flux Web (HTTP/HTTPS) sortants de l'infrastructure vers Internet. Il est hébergé sur une machine Fedora située en DMZ OUTBOUND (VLAN 101, 10.1.101.6) et la technologie utilisé est Squid.

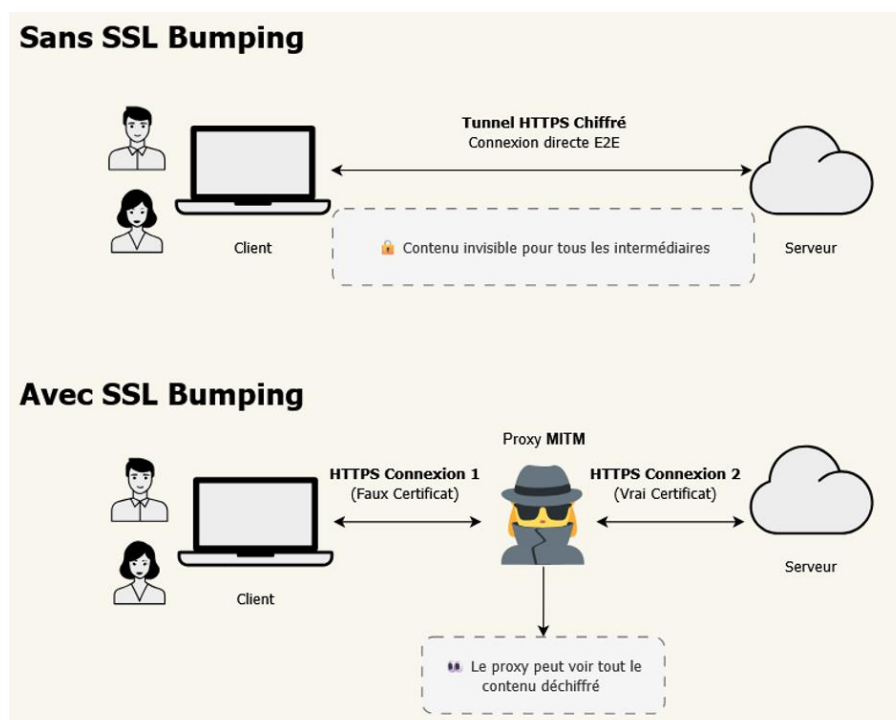
Il a un rôle de

- Filtrage (url malveillantes, ou autorisation strictement dédiée à installation de paquet ciblés)
- Traçabilité (Authentification des utilisateurs avec Kerberos)
- Inspection TLS (via SSL bumping pour déchiffrer du https)
- Cache (pour accélérer la navigation et économiser la bande passante)

Inspection HTTPS (SSL bumping)

Le trafic HTTPS étant chiffré, un proxy minimaliste ne voit que le nom du site visité (via le SNI du ClientHello), jamais le contenu ni l'URL complète. Le SSL bumping résout ce problème en plaçant le proxy en position de Man in the Middle mais de façon contrôlée :

- Le proxy établit lui-même la connexion TLS avec le vrai site de destination ;
- il génère en temps réel un faux certificat signé par la CA intermédiaire avec un DN correspondant au nom du site de destination
- le client, ayant la CA dans son trust store, met en place sa session TLS avec le proxy en pensant la faire avec le site
- le proxy peut ainsi lire le trafic en clair entre les deux sessions.



Il peut être décidé que les domaines jugés sensibles (banques, santé...) ne soient jamais déchiffrés : ils sont listés dans une ACL « nobump » (dans le squid.conf) et le proxy se contente uniquement de relayer le flux TLS chiffré en état sans le consulter. Les certificats générés sont mis en cache dans une base dédiée pour être réutilisés.

Modes de fonctionnement

Le proxy fonctionne selon deux modes, suivant le type de machine cliente :

Critère	Mode explicite	Mode transparent
Principe	Le client est configuré pour envoyer son trafic directement au proxy.	Le pare-feu (OPNsense) intercepte le trafic Web et le redirige par NAT vers le proxy, sans que le client le sache.
Machines concernées	Postes utilisateurs	Serveurs
Configuration et authentification	- Configuration des client déployable par GPO. - Authentification Kerberos au proxy via une keytab générée par le DC pour le compte de service du proxy. - Traçabilité par utilisateur.	- Aucune configuration - Aucune authentification - Traçabilité par IP uniquement.
Ports Squid	3128 (HTTP et HTTPS)	3127 (HTTP intercepté), 3129 (HTTPS intercepté)

Fichiers de configuration principaux

- **/etc/squid/squid.conf** : C'est le fichier principal du service, il définit les ACL (qui a le droit d'aller où), les trois ports d'écoute (3128, 3127, 3129), les règles de SSL bumping, l'authentification Kerberos et le cache et la journalisation.
- **/etc/squid/blocked_domains.txt** : C'est le fichier contenant la liste noire des domaines interdits, modifiable sans toucher à la configuration principale.
- **/etc/squid/ssl_cert/** : clé privée du proxy et certificat signé par la CA intermédiaire
- **/var/spool/squid/ssl_db** : base des certificats générés à la volée pour le SSL bumping.
- **/var/log/squid/access.log** : journal des accès internet (source, utilisateur, site, réponse d'accès).

Flux réseau OPNsense (firewall interne)

Source	Protocol	Ports source	Destination	Ports dest	Description
Postes utilisateurs	TCP	*	Proxy	3128	Navigation Web en mode explicite (HTTP et HTTPS)
Serveurs	TCP (HTTP)	*	Proxy (redirection NAT OPNsense)	3127	Flux HTTP (port 80) interceptés en mode transparent
Serveurs	TCP (HTTPS)	*	Proxy (redirection NAT OPNsense)	3129	Flux HTTPS (port 443) interceptés en mode transparent
Proxy	TCP	*	Internet	80, 443	Sortie Web pour le compte des clients
Proxy	TCP/UDP	*	DNS résolveur externe (10.1.101.5)	53	Résolution des domaines demandés par les clients
Postes utilisateurs	TCP/UDP	*	Active Directory	88	Obtention du ticket Kerberos présenté au proxy

HTTP Explicite (port 3128) flux détaillés

1. Client => Proxy:3128 : Le client envoie sa requête GET directement au proxy sans authentification « Proxy-Authorization » pour le moment :

*GET http://google.com/ HTTP/1.1
+ en-tête http sans le ticket "Proxy-Authorization"*

2. Proxy => Client:3128 : Le proxy lui demande d'ajouter ses identifiants avec "407 Proxy Authentication Required"

3. Client => RODC:88 => Client:88 => Proxy:3128 : La machine client demande et récupère le ticket de service du proxy auprès du RODC. Ce ticket est chiffré AES256 avec la clé dérivée du mot de passe du compte de service du proxy. Le client envoie ce ticket au proxy via la même requête GET dans l'en tête « Proxy-Authorization » :

*GET http://google.com/ HTTP/1.1
Proxy-Authorization: Negotiate <ticket Kerberos>
+ reste de l'en-tête http*

4. Proxy : Le proxy déchiffre le ticket Kerberos grâce à sa keytab locale qui contient la même clé AES ayant servi à chiffrer le ticket (clé dérivée du mot de passe du compte de service du proxy). Ensuite, il valide le ticket Kerberos et lit l'identité du client puis consulte les permissions relatives à ce compte dans squid.conf et décide d'autoriser ou refuser la connexion.

5. Proxy => google.com:80 => Proxy:80 => Client:3128 : Si autorisé, requête transmise à Google via GET puis réponse transmise au client (200 Connection Established) et si refusé, "erreur 403".

HTTP Transparent (port 3127) flux détaillés

1. Client => google.com:80 (OPNsense:80) :

*GET / HTTP/1.1 # ("/" = racine du site)
Host: google.com
+ reste de l'en-tête*

Le client croit parler directement à google.com d'où la destination incomplète et l'absence du nom de domaine dans le GET.

2. OPNsense => Proxy:3127 : OPNsense intercepte le paquet TCP destiné au port 80 puis réécrit la destination vers 10.1.101.6:3127 (géré via une règle NAT)

3. Proxy : Le proxy reçoit la requête lit Host: google.com et l'URL relative "/", reconstitue "http://google.com/" grâce au Host de l'en-tête, consulte les permissions relatives à cette IP dans squid.conf puis décide d'autoriser ou refuser

4. Proxy => google.com:80 => Proxy:80 => Client:80 : Si autorisé, requête transmise à Google via GET puis réponse transmise au client (200 Connection Established) et si refusé, erreur 403.

HTTPS explicite (port 3128) en SSL bumping flux détaillés

1. Client => Proxy:3128 :

```
CONNECT google.com:443 HTTP/1.1  
+ en-tête
```

Le client sait qu'il parle au proxy, d'où la destination proxy:3128, pas de ClientHello TLS ici, le CONNECT est en HTTP (non chiffré).

2. Proxy => Client:3128 : Le proxy lui demande d'ajouter ses identifiants avec 407 Proxy Authentication Required

3. Client => RODC:88 => Client:88 => Proxy:3128 : La machine client demande et récupère le ticket de service du proxy auprès du RODC. Ce ticket est chiffré AES256 avec la clé dérivée du mot de passe du compte de service du proxy. Le client envoie ce ticket au proxy via la même requête CONNECT dans l'en-tête « Proxy-Authorization » :

```
CONNECT google.com:443 HTTP/1.1  
+ "Proxy-Authorization: Negotiate <ticket Kerberos>"  
+ reste de l'en-tête http
```

4. Proxy : Le proxy déchiffre le ticket Kerberos grâce à sa keytab locale qui contient la même clé AES ayant servi à chiffrer le ticket (clé dérivée du mot de passe du compte de service du proxy). Ensuite, il valide le ticket Kerberos et lit l'identité du client puis consulte les permissions relatives à ce compte dans squid.conf et décide d'autoriser ou refuser la connexion.

5. Proxy => Client:3128 : Le proxy établit le tunnel TCP ouvert entre client et lui-même si autorisé (200 Connection Established), erreur 403 sinon.

6. Client => Proxy:3128 : Initialisation Handshake TLS : Le client envoie le ClientHello TLS composé du SNI = google.com

7. Proxy : Le proxy confirme le SNI "google.com" (déjà connu via le CONNECT de l'étape 1). Vérifie dans squid.conf si ce domaine n'est pas black-listé puis décide de bumper

8. Proxy <=> google.com:443 : Le proxy ouvre sa propre connexion TLS vers google.com, établit le handshake avec le vrai google.com, reçoit et lit le vrai certificat de google.com. Une session TLS Proxy <=> google.com est donc créée.

9. Proxy : Le proxy génère un faux certificat pour google.com avec le CN et les SANs du vrai certificat de google.com et le signe (proxy appartient à la chaîne de confiance de notre CA racine)

10. Proxy => Client:3128 : complète le handshake TLS avec le faux certificat. Le client voit CN = google.com (correspond) et signé par CA INT (présente dans son trust store), il pense donc parler à google.com et non au proxy. Une 2e session TLS Client <=> Proxy est créée.

11. Client => Proxy3128 : La session TLS étant créée, le client envoie sa requête HTTP déchiffrée :

```
GET / HTTP/1.1
Host: google.com
+ reste de l'en-tête
```

12. Proxy : lit Host: google.com et l'URL relative /, reconstitue https://google.com/ (même s'il l'avait déjà), consulte les permissions relatives au compte dans squid.conf puis décide d'autoriser ou refuser

13. Proxy => google.com:443 => Proxy:443 => Client:3128 : Si autorisé, requête transmise à Google via GET puis réponse transmise au client (200) et si refusé, erreur 403.

HTTPS transparent (port 3129) en SSL bumping flux détaillés

1. Client => google.com:443 : Le client initie un Handshake et envoie le ClientHello TLS composé du "SNI = google.com" Le client croit parler directement à google.com d'où la destination

2. OPNsense => Proxy 3129: intercepte le paquet TCP destiné au port 443 puis réécrit la destination vers le proxy:3129 (géré via une règle NAT)

3. Proxy : reçoit la connexion TCP, lit le ClientHello, extrait le SNI "google.com", vérifie dans le squid.conf si ce domaine n'est pas bloqué, puis décide de bumper

4. Proxy <=> google.com:443 : ouvre sa propre connexion TLS vers google.com, établit le handshake avec le vrai google.com, reçoit et lit le vrai certificat de google.com. Une session TLS Proxy <=> google.com est donc créée.

5. Proxy : génère un faux certificat pour google.com avec le CN et les SANs du vrai certificat de google.com et le signe lui-même (le proxy appartient à la chaîne de confiance de notre CA racine)

6. Proxy => Client:443 : Le proxy complète le handshake TLS avec le client grâce au faux certificat généré. Le client voit CN = google.com (correspond) et signé par CA INT (présente dans son trust store), il pense donc parler à google.com et non au proxy. Une 2e session TLS Client <=> Proxy est créée.

7. Client => OPNsense:443 => Proxy:3129 : Le client envoie sa requête HTTP GET au proxy :

*GET / HTTP/1.1
Host: google.com
+ reste de l'en-tête*

8. Proxy : reçoit la requête, lit Host: google.com et l'URL relative /, reconstitue "http://google.com/" grâce au Host de l'en-tête, consulte les permissions relatives à cette IP dans squid.conf puis décide d'autoriser ou refuser

9. Proxy => google.com:443 => Proxy:443 => Client:443 : Si autorisé, requête transmise à Google via GET puis réponse transmise au client (200) et si refusé, erreur 403.

Bastion (abandonné)

Le bastion sert à centraliser toutes les connexions / accès admin vers une infrastructure avant de la rediriger vers les machines de l'infrastructure.

Cela permet notamment :

- Collecter les logs d'administration de manière centralisée (traçage des connexions, accès strict et contrôlé, etc.)
- Simplifier la surveillance des sessions d'administration (enregistrement vidéo des sessions)
- MFA, authentification forte à plusieurs facteurs avant de pouvoir administrer des machines

Choix techniques

Composant	Justification
Apache Guacamole	Interface Web accessible sur une machine d'administration afin de gérer et configurer le bastion.
Tomcat9	Serveur Java local qui permet d'héberger et d'exécuter Guacamole. Cette version car elle marche mieux avec guacamole.
Java-21	Version 21 car sinon ça ne marche pas avec tomcat 9.
Guacd	Proxy natif qui assure la connexion distante entre le bastion et les autres machines.

MariaDB	Base de données afin de stocker les informations de l'application.
---------	--

Fonctionnement

A venir

Fichiers de configuration principaux

- /etc/guacamole/guacamole.properties : Fichier principal de la configuration Guacamole
- /etc/guacamole/user-mapping.xml : Fichier de la configuration des user / mdp de guacd
- /etc/guacamole/guacd.conf : Fichier de configuration de guacd
- /etc/guacamole/logback.xml : Fichier de configuration des logs Guacamole
- /etc/tomcat/server.xml : Fichier de la configuration du serveur Tomcat
- /var/lib/tomcat/webapps/guacamole.war : Application web Guacamole qui est déployée et hébergée dans Tomcat
- /etc/my.cnf : Fichier principal de MariaDB
- /etc/my.cnf.d/mariadb-server.cnf + /etc/my.cnf.d/client.cnf : Fichiers qui contiennent les paramètres du serveur et du client de MariaDB

Ansible

Ansible est l'outil de gestion centralisée des serveurs Linux de notre infrastructure. Il est hébergé sur la machine "ansible" (VLAN 70 ADDS, 10.0.70.6) et permet d'exécuter des actions identiques sur tout ou une partie du parc depuis un point unique : installation de paquets (mises à jour ou autres), modification de configuration et surtout hardening des machines.

Rôle

- Il permet d'appliquer de manière automatisée des configurations, des commandes, des actions récurrentes à toutes les machines ciblées de notre parc, ce qui évite de s'y connecter une par une et fait et nous fournit un gain de temps énorme. Il permet notamment le hardening de VMs, la mise en place de mises à jour régulière, d'installations de paquet...

Choix techniques

Composant	Solution	Justification
Gestion de configuration	Ansible (ansible-core)	Standard open-source sans agent : seule une connexion SSH est nécessaire côté cible. Playbooks YAML lisibles et rejouables.
Connexion aux cibles	SSH par clé + compte de service de nom « ansible » sur chaque machine cible	Authentification par clé uniquement. La clé privée ne quitte jamais le serveur Ansible. Le compte de service dispose de sudo sans mot de passe, nécessaire à l'automatisation.
Protection des secrets	Ansible Vault	Les credentials utilisés par les playbooks sont stockés dans des fichiers chiffrés : aucun mot de passe en clair dans les playbooks.

Fonctionnement

Ansible se connecte en SSH aux machines déclarées dans son inventaire et y exécute les tâches des playbooks. L'inventaire (hosts.yml) est hiérarchisé : les machines y sont classées par OS (Fedora / Debian), par type (serveur / workstation), puis par criticité (critique / important / autres) et enfin par service.

Un playbook se lance ainsi, depuis le répertoire `~/ansible/` :

`ansible-playbook <playbook>.yml`

L'ajout de machines Windows est faisable via WinRM (HTTPS, port 5986) mais se fera si le besoin s'y fait ressentir.

Fichiers de configuration principaux

- `~/ansible/ansible.cfg` : configuration globale d'Ansible. On y définit le chemin de l'inventaire, utilisateur distant, clé privée SSH, fichier de mot de passe Vault...
- `~/ansible/inventory/hosts.yml` : inventaire des machines gérées, organisé en groups.
- `~/ansible/group_vars/` et `host_vars/` : variables propres à un groupe ou à une machine précise.
- `~/ansible/playbooks/` : l'ensemble des playbooks.
- `~/.ssh/id_ed25519` : clé privée SSH du serveur Ansible qui ne quitte jamais cette machine.
- `~/.vault_pass` : mot de passe maître d'Ansible Vault (en lecture seule).

Flux réseau

Source	Protocol	Ports	Destination	Ports	Description
Ansible (10.0.70.6)	TCP	*	Parc Linux	22	Connexion SSH d'administration (exécution des playbooks)
Parc Linux	TCP/UDP	80/443	*	*	Pour l'installation des paquets pour le hardening

Serveur de fichiers

Le serveur de fichiers centralise le stockage et le partage des documents de l'entreprise.

Choix techniques

Composant	Solution	Justification
Service de partage	Samba	Standard du domaine. Parfaite compatibilité avec les postes Windows

Fichiers de configuration principaux

- `/etc/samba/smb.conf` : fichier de configuration du service de partage. Définis deux espaces distincts (Bureautique et Confidentiel) avec leurs restrictions d'accès associées.
- `/etc/krb5.conf` : Configuration du client Kerberos. Assure la jonction avec le contrôleur de domaine de l'infrastructure.

Flux réseau

Source	Protocol	Ports	Destination	Ports	Description
Serveur de fichiers (10.0.20.10)	TCP/UDP		DC (10.0.70.5)	88, 389, 464	Authentification (Kerberos), requêtes d'annuaire (LDAP) et vérification des droits

Miroir (abandonné)

Un miroir local est une copie, hébergée dans le LAN, des dépôts officiels Fedora. Grace à ce système, les machines du parc (surtout celles ne devant pas avoir accès à internet) pouvaient installer leurs paquets et mises à jour depuis ce serveur interne via HTTP. Une seule machine (le miroir) a besoin d'un accès Internet vers les dépôts. Cela avait plusieurs avantages :

- **Bande passante** : un seul téléchargement WAN par paquet, quelle que soit la taille du parc, puis distribution en LAN. Utile pour un parc qui aurait pu s'étendre comme l'évoquait le sujet.
- **Disponibilité hors Internet** : si le lien WAN tombe, les installations et mises à jour restent possibles.
- **Une surface d'attaque réduite** : une seule règle pare-feu (VLANs internes => 10.0.30.5:80) au lieu d'un grand nombre d'autorisations sortantes servant uniquement à ça.
- **Décharger le proxy** : Planifier les mises à jour pour toutes les VMs en même temps risque de saturer le proxy, d'autant plus que le parc est amené à s'agrandir. Le miroir aurait réglé ce problème

Nous savions par contre dès le début que cette solution allait demander beaucoup de disque (environ 165 Go mesurés pour une seule version de Fedora) et qu'elle serait plus longue à mettre en place qu'un simple passage par le proxy.

Le système a été automatisé avec Ansible et testé. Il fonctionnait pour les VLANs internes. Nous avons fini par l'arrêter pour trois raisons que nous allons évoquer.

Architecture mise en place

Le miroir tournait sur une VM à 2 vCPU, 2 Go de RAM et 250 Go de disque (oui c'est beaucoup mais nous avons défini un disque dynamique prenant en compte uniquement ce qui est consommé). Il exposait deux URLs en HTTP, <http://10.0.30.5/mirror/updates/> pour les mises à jour et <http://10.0.30.5/mirror/fedora/> pour les nouveaux paquets. La synchronisation avec les dépôts officiels se faisait chaque nuit à minuit, et les clients se mettaient à jour quotidiennement à 4h. Tout cela était monitoré par des playbooks Ansible contenant des timers systemd.

Pour ce qui est de la volumétrie, on était sur du 110 Go pour le dépôt fedora et 54 Go pour le dépôt updates (environ 164 Go en tout pour 2h30 de synchronisation la 1^{ère} fois).

Fonctionnement

Sur le miroir, la commande « `dnf reposync` » téléchargeait les paquets des dépôts officiels en les plaçant dans « `/var/www/html/mirror` » puis « `createrepo_c` » générait

les métadonnées associés à ce téléchargement pour que dnf puisse lire ces paquets. Apache exposait ensuite ces dossiers en HTTP. Sur chaque client, deux fichiers .repo pointaient vers le miroir et un fichier d'override désactivait les dépôts publics, pour que seuls les dépôts locaux soient utilisés.

Causes de l'abandon

Trois éléments nous ont poussés abandonner cette idée.

- Un problème réseau toujours non résolu à ce jour. Les machines des VLANs internes joignaient bien le miroir, mais aucune machine en DMZ n'y arrivait. On a pu observer que le client envoyait sa demande correctement, que le miroir répondait, mais que cette réponse se perdait au retour, au moment de traverser l'OPNsense. La seule configuration qui marchait était celle de tout laisser passer, ce qui n'est pas envisageable de garder en production. Dès qu'on essayait de la resserrer cette règle, le flux recassait.
- La contrainte sur les DNS. Ayant mal lu le sujet, nous avons pris connaissance en soutenance que celui-ci imposait du Debian (ou freebsd) pour les serveurs DNS. Le miroir Fedora n'aurait alors plus suffi et il aurait fallu y ajouter les dépôts Debian, soit environ deux fois plus de disque, alors que la place utilisée était déjà assez conséquente.
- Une solution plus simple existait. Celle de faire passer les machines sans accès Internet par le proxy, avec une politique qui bloque tout sauf les URLs des dépôts officiels (mises à jour toujours via des playbooks).

Retour en arrière

Un playbook de rollback a donc été passé sur tout le parc pour revenir à la situation d'avant le miroir (réactivation des dépôts publics fedora et updates, suppression des dépôts pointant vers le miroir et de l'override et arrêt des timers).

ERP ODOO

L'ERP permet de centraliser la gestion de l'entreprise afin de faciliter les tâches et les processus pour les employés. Celui-ci tourne sur Odoo, un ERP opensource modulaire, ce qui signifie qu'on peut installer et déployer que les modules de l'ERP dont l'entreprise a besoin.

Choix techniques

Composant	Solution	Justification
Service ERP	Odoo	Opensource modulaire, donc adaptable à l'entreprise.
Conteneurisation de l'ERP	Docker Compose	La version la plus récente accessible de Odoo (18.0) n'est pas compatible avec Fedora 43 et Debian 13 Trixie. Pour les faire tourner sur ces machines serveurs, il faut passer par Docker Compose.
Base de données	PostgreSQL	La base de données relationnelle requise par défaut par Odoo.

Fichiers de configuration principaux

- /home/odoo-admin/odoo/docker-compose.yml : fichier de configuration du service docker compose qui permet de lancer Odoo et sa base de données.
- 2 conteneurs créés par le fichier yaml ci-dessus : odoo-db et odoo-app, qui contiennent respectivement la base de données odoo et l'application web / le service odoo.

Flux réseau

Source	Protocol	Ports	Destination	Ports	Description
TCP/UDP	any	10.0.31.2	http/https	80, 443	Exposer interface de ERP